ASSIGNMENT-19.2

Name: Chirra Jyothika

RollNo: 2303A51280

BatchNo: 05

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

• Prompt AI to translate a Python script that reads from and writes to a file into Java.

• Ensure proper handling of file streams, exceptions, and input/output formatting.

• Verify that the Java program produces the same file content and console output as the Python version.

• Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Prompt:

```
Generate an equivalent Java program for the above Python code that performs file handling operations such as reading from a file and writing processed content to another file. Ensure the use of appropriate Java classes like BufferedReader and BufferedWriter, include proper exception handling, and maintain the same functionality and output format as the Python program.
```

Code:

Python

```python
# Write to a file
with open("input.txt", "w") as file:
    file.write("Hello, this is a sample file.\n")
    file.write("This is the second line.")

# Read from the file
with open("input.txt", "r") as file:
    content = file.read()

print("File Content:")
print(content)
```

```java
import java.io.*;

public class FileHandler {
    Run | Debug
    public static void main(String[] args) {
        String inputFile = "input.txt";
        StringBuilder content = new StringBuilder();

        // Write to a file
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(inputFile))) {
            writer.write(str: "Hello, this is a sample file.\n");
            writer.write(str: "This is the second line.");
        } catch (IOException e) {
            System.out.println("Error writing to file: " + e.getMessage());
            return;
        }

        // Read from the file
        try (BufferedReader reader = new BufferedReader(new FileReader(inputFile))) {
            String line;
            while ((line = reader.readLine()) != null) {
                content.append(line).append(System.lineSeparator());
            }
        } catch (FileNotFoundException e) {
            System.out.println(x: "Error: Input file not found.");
            return;
        } catch (IOException e) {
            System.out.println("Error reading from file: " + e.getMessage());
            return;
        }

        // Print file content
        System.out.println(x: "File Content:");
        System.out.print(content.toString());
    }
}
```

Output:

```
File Content:
Hello, this is a sample file.
This is the second line.

Data written to output.txt successfully.
```

Explanation:

- The program performs file handling by reading data from an input file and writing to an output file.

- It uses the open() function with read ("r") and write ("w") modes.
- The file content is displayed on the console for verification.
- The content is processed by converting it into uppercase.
- The processed data is written into a new file (output.txt).
- Exception handling (try-except) is used to handle errors like missing files or unexpected issues.
- This ensures the program runs safely without crashing and provides meaningful error messages.

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into

Python using AI assistance.

Instructions:

• Provide AI with a C++ implementation of a sorting technique

(e.g., Bubble Sort or Selection Sort).

• Ask AI to generate an equivalent Python implementation.

• Ensure logical correctness and validate results using multiple

test cases.

• Confirm that the Python version maintains the same sorting

behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++

sorting implementation.

Prompt:

```
Generate an equivalent Python program for the given C++
Bubble Sort implementation. The program should take user
input for the number of elements and the array values,
perform sorting using the Bubble Sort algorithm, and display
the sorted array. Ensure that the logic and behavior remain
the same as the C++ version and the code is clean, readable,
and executable.
```

Code:

```cpp
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                // swap elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements:\n";
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    bubbleSort(arr, n);

    cout << "Sorted array:\n";
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

def main():
    try:
        n = int(input("Enter number of elements: "))
        arr = list(map(int, input("Enter elements:\n").split()))
        if len(arr) != n:
            print(f"Error: Expected {n} elements, got {len(arr)}.")
            return
        bubble_sort(arr)
        print("Sorted array:")
        print(' '.join(map(str, arr)))
    except Exception as e:
        print("Error:", e)

if __name__ == "__main__":
    main()
```

Output:

```
Enter number of elements: 5
Enter elements:
3 5 1 2 3
Sorted array:
1 2 3 3 5
```

Explanation:

- The C++ program is used to sort numbers.

- The same sorting logic is converted into Python.
- Both programs should give the same sorted output.
- The Python code should be simple and easy to understand.

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

• Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).

• Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.

• Test the generated code using a sample dataset.

• Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Prompt:

```
Convert this SQL query into equivalent Pandas code. The Pandas program
should do the same work as the SQL query and produce the same result.
Use groupby() and aggregation functions like sum(), count(), mean(), or
any other needed function. Keep the code simple and easy to understand.
Also explain the Pandas code in simple language and test it with a sample
dataset to make sure the output matches the SQL query. Give the final
answer with SQL query, Pandas code, sample dataset, explanation, sample
output, and test case.
```

Code:

```python
import pandas as pd

# Define the employees DataFrame
employees = pd.DataFrame({
    'department': ['HR', 'HR', 'Sales', 'Sales', 'IT'],
    'sales': [100, 150, 200, 250, 300]
})

result = (
    employees
    .groupby('department', as_index=False)
    .agg(total_sales=('sales', 'sum'), avg_sales=('sales', 'mean'))
)

print(result)
```

Output:

```
  department  total_sales  avg_sales
0         HR          250      125.0
1         IT          300      300.0
2      Sales          450      225.0
```

Explanation:

- SQL is used to work with data in tables.
- GROUP BY is used to group similar values together.
- Aggregate functions are used to calculate values like total, average, count, etc.
- In Pandas, the same thing is done using groupby() and aggregation functions.
- The final result in Pandas should match the SQL query output.

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario:

Students will select a practical real-world application requiring the same

logic in two different programming languages.

Example:

• Converting a REST API implementation from Python (Flask) to

Java (Spring Boot).

• Translating a data processing script from Python to C++ for

performance optimization.

Instructions:

• Provide AI with source code in one programming language.

• Prompt AI to convert the code into another chosen language.

• Execute and test both versions to ensure identical functionality.

• Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Prompt:

> Provide the following Python Flask REST API code and convert it into an equivalent Java Spring Boot application. Ensure that the functionality remains the same, including the endpoint /hello and the JSON response. The generated Java code should be complete, executable, and follow proper Spring Boot structure. After conversion, verify that both implementations produce identical outputs

Code:

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/hello', methods=['GET'])
def hello():
    return jsonify({'message': 'Hello, World!'})

if __name__ == '__main__':
    app.run(debug=True)
```

Gemini

```python
import os
import subprocess
import threading
import time
from flask import Flask, jsonify

# --- Flask Application Setup ---
# The Flask application code is redefined here to be run in a subprocess.
# This ensures it doesn't block the cell's execution and can be tested alongside Spring Boot.

print("Starting Flask application in background...")

flask_app_code = """from flask import Flask, jsonify;\napp = Flask(__name__);\n@app.route(\"/hello\", methods=[\"GET\"]);\ndef hello(): return jsonify({\"message\": \"Hello, World!\"});\napp.run(debug=False, port=5000)"""

flask_process = None
try:
    # Use subprocess to run Flask, so it doesn't block the main thread
    # Redirect output to /dev/null to keep cell output clean
    flask_process = subprocess.Popen(['python', '-c', flask_app_code],
                                     stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, preexec_fn=os.setsid)
    print(f"Flask app started with PID: {flask_process.pid}")
except Exception as e:
    print(f"Failed to start Flask app: {e}")

time.sleep(1) # Give Flask app time to start

# --- Spring Boot Application Setup ---

print("\nSetting up Spring Boot project structure...")
!mkdir -p src/main/java/com/example/demo
!mkdir -p src/main/resources

print("\nCreating pom.xml...")
# Using standard file I/O to create pom.xml
with open("pom.xml", "w") as f:
    f.write("""<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<project xmlns=\"http://maven.apache.org/POM/4.0.0\" xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\"\n\txsi:schemaLocation=\"http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd\">\n\t<modelVers

print("Creating Application.java...")
# Using standard file I/O to create Application.java
with open("src/main/java/com/example/demo/Application.java", "w") as f:
    f.write("""package com.example.demo;\n\nimport org.springframework.boot.SpringApplication;\nimport org.springframework.boot.autoconfigure.SpringBootApplication;\n\n@SpringBootApplication\npublic class Application {\n\n\tpublic static void main(String[] args) {\n\t\tSpringApplication


print("Creating HelloController.java...")
# Using standard file I/O to create HelloController.java
with open("src/main/java/com/example/demo/HelloController.java", "w") as f:
    f.write("""package com.example.demo;\n\nimport java.util.HashMap;\nimport java.util.Map;\n\nimport org.springframework.web.bind.annotation.GetMapping;\nimport org.springframework.web.bind.annotation.RestController;\n\n@RestController\npublic class HelloController {\n\n    @GetMappin

# --- Compile and Run Spring Boot Application ---

print("\nInstalling Maven if not present...")
# This will only install if not present, and it's silent
!apt-get update > /dev/null 2>&1
!apt-get install -y maven > /dev/null 2>&1

print("Compiling Spring Boot application...")
# Clean and package the Spring Boot application into an executable JAR
# -DskipTests is used to skip running tests during package phase
compile_result = subprocess.run(['mvn', 'clean', 'package', '-DskipTests'], capture_output=True, text=True)
if compile_result.returncode != 0:
    print("Maven compilation failed:")
    print(compile_result.stdout)
    print(compile_result.stderr)
    # Attempt to clean up Flask process before raising error
    if flask_process and flask_process.poll() is None:
        os.killpg(os.getpgid(flask_process.pid), 9)
    raise RuntimeError("Spring Boot compilation failed.")
else:
    print("Spring Boot application compiled successfully.")

spring_boot_process = None
spring_boot_pid = None

print("Starting Spring Boot application in background...")
```

```python
# Find the generated JAR file
jar_files = [f for f in os.listdir('target') if f.endswith('.jar') and 'original' not in f]
if not jar_files:
    if flask_process and flask_process.poll() is None:
        os.killpg(os.getpgid(flask_process.pid), 9)
    raise FileNotFoundError("JAR file not found in target/. Did Maven package successfully?")

jar_path = os.path.join('target', jar_files[0])

# Start the Java process
try:
    spring_boot_process = subprocess.Popen(['java', '-jar', jar_path],
                                           stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, preexec_fn=os.setsid)
    spring_boot_pid = spring_boot_process.pid
    print(f"Spring Boot application started with PID: {spring_boot_pid}")
except Exception as e:
    print(f"Failed to start Spring Boot app: {e}")
    if flask_process and flask_process.poll() is None:
        os.killpg(os.getpgid(flask_process.pid), 9)
    raise

time.sleep(15) # Give Spring Boot server time to start

print("Spring Boot application should be running.")

# --- Test Cases and Verification ---

print("\n--- Testing Flask API (Port 5000) ---")
flask_test_result = subprocess.run(['curl', 'http://127.0.0.1:5000/hello'], capture_output=True, text=True)
print(flask_test_result.stdout.strip())
if flask_test_result.returncode != 0:
    print(f"Flask test failed: {flask_test_result.stderr.strip()}")

print("\n--- Testing Spring Boot API (Port 8080) ---")
spring_boot_test_result = subprocess.run(['curl', 'http://localhost:8080/hello'], capture_output=True, text=True)
print(spring_boot_test_result.stdout.strip())
if spring_boot_test_result.returncode != 0:
    print(f"Spring Boot test failed: {spring_boot_test_result.stderr.strip()}")

# --- Clean up Processes ---

print("\n--- Cleaning up processes ---")
if flask_process and flask_process.poll() is None: # Check if still running
    print(f"Terminating Flask application with PID: {flask_process.pid}")
    try:
        os.killpg(os.getpgid(flask_process.pid), 9) # SIGKILL
        print("Flask application terminated.")
    except OSError as e:
        print(f"Error terminating Flask process {flask_process.pid}: {e}")
else:
    print("Flask PID not found or already terminated.")

if spring_boot_process and spring_boot_process.poll() is None: # Check if still running
    print(f"Terminating Spring Boot application with PID: {spring_boot_pid}")
    try:
        os.killpg(os.getpgid(spring_boot_pid), 9) # SIGKILL
        print("Spring Boot application terminated.")
    except OSError as e:
        print(f"Error terminating Spring Boot process {spring_boot_pid}: {e}")
else:
    print("Spring Boot PID not found or already terminated.")

print("\nVerification complete.")
```

Output:

```
Starting Flask application in background...
Flask app started with PID: 4379

Setting up Spring Boot project structure...

Creating pom.xml...
Creating Application.java...
Creating HelloController.java...

Installing Maven if not present...
Compiling Spring Boot application...
Spring Boot application compiled successfully.
Starting Spring Boot application in background...
Spring Boot application started with PID: 5587
Spring Boot application should be running.

--- Testing Flask API (Port 5000) ---

Flask test failed: % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
curl: (7) Failed to connect to 127.0.0.1 port 5000 after 0 ms: Connection refused

--- Testing Spring Boot API (Port 8080) ---


--- Cleaning up processes ---
Flask PID not found or already terminated.
Spring Boot PID not found or already terminated.

Verification complete.
```

Explanation:

- The source code is written in one programming language.
- The same logic is converted into another programming language.
- Both versions should perform the same task.
- Both programs are tested with the same input.
- Differences in syntax, libraries, and execution are observed and documented.

Task -5: (Basic Function Translation (Python to C++)

Translate a simple Python function that checks whether a number is even

or odd into C++ using AI assistance.

Instructions:

• Provide AI with a basic Python function that determines if a

number is even or odd.

• Prompt AI to generate equivalent C++ code.

• Ensure correct syntax, input handling, and output formatting.

• Compile and run the C++ program to verify it produces the same result as the Python version.

• Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python function, correctly identifying whether a number is even or odd for all test cases.

Prompt:

Convert this Python program into C++. The C++ program should do the same work as the Python program and follow the same logic. The program should check whether a number is even or odd. Keep the C++ code simple, correct, and easy to understand. Make sure it handles input properly and gives the correct output in the same format. Also explain the C++ code in simple language and test it with at least three different input values to verify that both programs give the same result. Give the final answer with Python code, C++ code, explanation, sample input, sample output, and test cases.

Code:

t5(19.2).py > check_even_odd

```python
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Input and output
num = int(input("Enter a number: "))
print(check_even_odd(num))
```

```
%%writefile even_odd.cpp
#include <iostream> // Required for input/output operations

// Function to check if a number is even or odd
void checkEvenOddCplusplus(int number) {
    // The modulo operator (%) returns the remainder of a division.
    // If a number divided by 2 has a remainder of 0, it's even.
    // Otherwise, it's odd.
    if (number % 2 == 0) {
        std::cout << number << " is Even" << std::endl; // Print "Even" to the console
    } else {
        std::cout << number << " is Odd" << std::endl;  // Print "Odd" to the console
    }
}

int main() {
    int num; // Declare an integer variable to store the user's input

    std::cout << "Enter an integer: "; // Prompt the user to enter a number
    std::cin >> num; // Read the integer entered by the user

    checkEvenOddCplusplus(num); // Call the function to check if the number is even or odd

    return 0; // Indicate successful execution of the program
}
```

```
print("\n--- Python Test Cases ---")
print(f"Input: 4  -> Output: {check_even_odd_python(4)}")
print(f"Input: 7  -> Output: {check_even_odd_python(7)}")
print(f"Input: 0  -> Output: {check_even_odd_python(0)}")
print(f"Input: -3 -> Output: {check_even_odd_python(-3)}")


print("\n--- C++ Compilation and Execution Example ---")
print("To compile and run the C++ code, you would typically use a g++ compiler in a terminal:")
print("  g++ even_odd.cpp -o even_odd")
print("  ./even_odd")

print("\n--- Expected C++ Outputs for the same inputs ---")
print("Input: 4  -> Output: 4 is Even")
print("Input: 7  -> Output: 7 is Odd")
print("Input: 0  -> Output: 0 is Even")
print("Input: -3 -> Output: -3 is Odd")
```

Output:

```
--- Python Test Cases ---
Input: 4  -> Output: 4 is Even
Input: 7  -> Output: 7 is Odd
Input: 0  -> Output: 0 is Even
Input: -3 -> Output: -3 is Odd

--- C++ Compilation and Execution Example ---
To compile and run the C++ code, you would typically use a g++ compiler in a terminal:
  g++ even_odd.cpp -o even_odd
  ./even_odd

--- Expected C++ Outputs for the same inputs ---
Input: 4  -> Output: 4 is Even
Input: 7  -> Output: 7 is Odd
Input: 0  -> Output: 0 is Even
Input: -3 -> Output: -3 is Odd
```

Explanation:

- The program takes a number as input.
- It checks whether the number is divisible by 2.
- If divisible, it prints Even.
- If not divisible, it prints Odd.
- The same logic is written in both Python and C++.